

LEZIONE 22

PROGETTAZIONE CON

RESPONSABILITÀ

&&

INTRODUZIONE AI PATTERN

Ingegneria del Software e Progettazione Web
Università degli Studi di Tor Vergata - Roma

Guglielmo De Angelis
guglielmo.deangelis@iasi.cnr.it

cosa è UML :

“In short, the Unified Modeling Language (UML) provides industry standard mechanisms for visualizing, specifying, constructing, and documenting software systems.”

cosa è UML

*“In short, the Unified Modeling Language (UML) provides industry standard mechanisms for **visualizing, specifying, constructing, and documenting software** systems.”*

- UML è un linguaggio
- UML non impone l'adesione ad uno specifico processo di sviluppo del software
- la conoscenza “sintattica” di UML non è funzione di un'appropriata applicazione dei principi O.O.

UML: mezzo non soluzione

- l'esperienza nella modellazione O.O. suggerisce
 - principi generali
 - soluzioni ricorrenti
 - aspetti da considerare in funzione:
 - del contesto
 - della fase del processo adottato
 - degli obiettivi da raggiungere
- tuttavia spesso è intesa come una “forma creativa” basata su abilità personali e attitudini derivanti dallo studio e dalla pratica
 - una forma di artigianato?!?!?

RDD – 1

- un modo di pensare alla progettazione O.O.
 - **R**esponsibility **D**riven **D**esign
- gli oggetti software sono associati ad una descrizione delle loro responsabilità
 - obblighi sulla struttura o sul comportamento di un oggetto in relazione al suo ruolo all'interno del sistema
 - sono assegnate alle classi ed agli oggetti dal progettista

RDD – 2

- definire elementi del sistema
 - responsabilità
 - di fare
 - istanziare un oggetto, computare un calcolo,
 - dare inizio alle attività di altri oggetti
 - controllare e coordinare parte delle attività di altri oggetti
 - di conoscere
 - i propri dati privati incapsulati
 - gli oggetti correlati
 - cose che possono essere derivate o calcolate
 - ruoli
 - collaborazioni

GRASP

- **General Responsibility Assignment Software Principles** (o Patterns)
 - principi di supporto all'apprendimento/progettazione O.O.
 - si basano sui concetti base di RDD
- GRASP vs GoF
 - GRASP sono principi generali che guidano le scelte progettuali attraverso l'assegnamento di responsabilità
 - GoF sono soluzioni tipiche a problemi ricorrenti (vedi lezioni 29 e 33)
 - idee di progettazione avanzate

GRASP

- **G**eneral **R**esponsibility **A**ssignment **P**atterns (o Principles (o Patterns)
 - principi di supporto
 - si basano su una relazione O.O.
- **non sempre è possibile l'applicazione contemporanea di tutti i principi discussi nel seguito. E' compito del progettista valutare le differenti soluzioni e scegliere cosa prediligere**
- scelte responsabilità
- problemi ricorrenti (vedi lezioni 29 e avanzate)

GRASP

- principali GRASP considerati :
 - creator
 - information expert
 - low coupling
 - high cohesion
 - single responsibility principle
 - non è considerato un GRASP in senso stretto
 - controller
 - principio conoscenza minima (legge di Demetra)
 - non è considerato un GRASP in senso stretto

GRASP

- principali GRASP considerati :
 - creator
 - information expert
 - low coupling
 - high cohesion
 - single responsibility principle
 - non è considerato un GRASP in senso stretto
 - controller
 - principio conoscenza minima (legge di Demetra)

potete usare questi principi per:

- **analizzare e valutare una soluzione che vi è stata esposta**
- **argomentare i motivi che vi hanno portato alla proposta di una determinata soluzione**

creator

- Problema
 - chi crea/istanzia un oggetto relativo ad una classe A?
- Soluzione
 - assegna alla classe B la responsabilità di creare un'istanza della classe A se una delle seguenti condizioni è vera
 - B compone/aggrega un insieme di oggetti di tipo A
 - B registra A presso un oggetto che ha la responsabilità di “contenitore” di istanze
 - B utilizza “in modo esclusivo” A, cioè solo istanze di tipo B sono responsabili di interrogare e/o modificare istanze di tipo A
 - B possiede tutti i dati per poter inizializzare A
 - la scelta tra soluzioni alternative dovrebbe essere fatta preferendo quelle soluzioni che massimizzano il numero di condizioni vere

information expert

- Problema
 - chi ha le informazioni per assolvere ad un determinato scopo?
 - chi ha la responsabilità di conoscere dettagli su come una funzionalità/responsabilità viene soddisfatta?
- Soluzione
 - assegna una responsabilità alla classe che possiede le informazioni necessarie per soddisfarle
 - capire il ruolo che una determinata classe sta svolgendo nel sistema e quali informazioni è in grado di recuperare/mantenere
 - definire degli “esperti di informazione” aiuta a trasformare associazioni in aggregazioni o composizioni, ed a dettagliare l'insieme di operazioni fornite da una classe/interfaccia

low coupling – 1

- accoppiamento: misura del grado di dipendenza esistente tra classi distinte di uno stesso sistema software
 - stima quanto fortemente l'implementazione di una classe è connessa ad altre, quanto ne conosce, o quanto dipende da esse
 - vedi lezioni 9, 10, 12, 13, 18, 19
- Problema
 - come ridurre l'impatto dei cambiamenti?
- Soluzione
 - assegna responsabilità in modo tale che l'accoppiamento (quello non necessario!!!!) rimanga basso
 - alto grado di accoppiamento → soluzione fortemente dipendente da scelte implementative
 - usa questo principio per valutare soluzioni alternative

low coupling – 2

- Esempi:
 - le forme più comuni di coupling tra `TypeX` e `TypeY`
 - `TypeX` ha un attributo `TypeY` o è associato a `TypeY`
 - `TypeX` richiama servizi di un oggetto `TypeY`
 - `TypeX` ha una operazione il cui metodo associato include il riferimento ad una istanza di `TypeY` (e.g. variabili locali, parametri o tipi di ritorno nei metodi invocati, etc)
 - `TypeX` è sottoclasse (diretta o indiretta) di `TypeY`
 - `TypeY` è un'interfaccia, che `TypeX` implementa
 - forma di coupling non sempre necessario
 - `TypeX` ha un attributo `attr1:TypeK` ed è associato a `TypeY`, `TypeY` offre una operazione `op1` in grado di processare istanze `TypeK`; `TypeX` non agisce direttamente su `attr1` interagendo con le operazioni di `TypeK` ma delega `TypeY.op1` per la sua modifica
- Nota:
 - information expert “sostiene” low coupling
 - “sostiene” = “condizione necessaria ma non sufficiente”

high cohesion – 1

- coesione: misura del grado di correlazione tra le operazioni di un elemento software dal punto di vista funzionale
 - stima quanto fortemente
 - la definizione di una classe sia concettualmente affine ad altre
 - le operazioni fornite dalla classe siano consistenti con le responsabilità associate alla classe
 - implica una misura del “carico di lavoro” potenziale (i.e. responsabilità) assegnato ad una classe
 - vedi lezioni 9, 10, 12, 13, 18, 19
- Problema
 - come mantenere oggetti focalizzati, comprensibili, gestibili e che implicino “low coupling”?
- Soluzione
 - assegna responsabilità in modo tale che la coesione rimanga alta
 - alto grado di coesione → soluzione ben strutturata nel dominio del problema da risolvere
 - usare classi che definiscono un insieme (relativamente) limitato di operazioni, riferenti ad operazioni altamente correlate
 - usa questo principio per valutare scelte alternative

high cohesion – 2

- esempi di gradi di coesione funzionale
 - coesione bassa
 - una classe è la sola responsabile di molte cose in aree diverse
 - una classe ha da sola la responsabilità di un compito complesso in una sola area funzionale
 - coesione alta
 - una classe ha responsabilità moderate in un'unica area funzionale e collabora con altre classi per svolgere i suoi compiti
 - coesione moderata
 - una classe A ha da sola la responsabilità di leggere in un insieme limitato e scorrelato di aree funzionali diverse che sono logicamente legate al concetto modellato da A

high cohesion – 3, metriche di esempio

- Cohesion at Message Level

$$CHM = \frac{1}{N} \sum_{i=1}^N chm_i$$

$$f_{msg}(opr_k, opr_m) = \frac{\left(\frac{|ret_k \cap ret_m|}{|ret_k \cup ret_m|} \right) + \left(\frac{|par_k \cap par_m|}{|par_k \cup par_m|} \right)}{2}$$

$$chm_i = \begin{cases} \frac{\sum_{(k,m)} f_{msg}(opr_k, opr_m)}{\frac{1}{2} |O_i| \times (|O_i| - 1)}, & \text{if } |O_i| \neq 1 \\ 1, & \text{if } |O_i| = 1 \end{cases}$$

O : insieme di operazioni della classe

- Cohesion at Domain Level

$$CHD = \frac{1}{N} \sum_{i=1}^N chd_i$$

$$f_{dom}(opr_k, opr_m) = \frac{|f_{term}(opr_k) \cap f_{term}(opr_m)|}{|f_{term}(opr_k) \cup f_{term}(opr_m)|}$$

$$chd_i = \begin{cases} \frac{\sum_{(k,m)} f_{dom}(opr_k, opr_m)}{\frac{1}{2} |O_i| \times (|O_i| - 1)}, & \text{if } |O_i| \neq 1 \\ 1, & \text{if } |O_i| = 1 \end{cases}$$

f_{term} : insieme dei termini semanticamente affini nel dominio; approssimazione della semantica dell'operazione

single responsibility principle – 1

- “A class should have one, and only one, reason to change.”
Robert C. Martin
- A class should provide well-defined, independent, and coherent set of functionality wrt the other classes that use it
 - mantenere minimo il numero di interfacce implementate da una classe: più piccolo l'insieme delle interfacce realizzate, più alta la probabilità che la classe si impegni a soddisfare un'unica responsabilità
 - NOTA: minimo NON SIGNIFICA che sia obbligatorio implementare sempre solo una interfaccia
 - mantenere l'insieme di operazioni definite in una classe coerente con l'insieme di responsabilità da essa assunte
- non è specificatamente un GRASP, ma ne è conseguenza diretta

single responsibility principle – 2

metriche di esempio

- Interface Number: numero di interfacce implementate da una classe
 - deve essere minimizzato all'essenziale

$$IFN = \frac{1}{N} \sum_{i=1}^N ifn_i$$

I : insieme di interfacce della classe

$$ifn_i = |I_i|$$

- Interaction Number: numero di chiamate tra classi differenti
 - deve essere contenuto alle interazioni essenziali

$$IRN = \sum_{(opr_j, opr_k)} w_{j,k}$$

controller

- Problema
 - chi ha la responsabilità di coordinare lo svolgimento di un insieme di attività (omogenee) svolte da un (sotto-)sistema?
- Soluzione
 - assegna la responsabilità a un oggetto che
 - rappresenti il punto di accesso al (sotto-)sistema di riferimento
 - applicazione del GoF Façade (vedi lezione 33)
 - raggruppi un insieme di operazioni volte a identificare uno specifico modo di utilizzo del (sotto-)sistema considerato (e.g. chi è responsabile della realizzazione di uno scenario per un caso d'uso)

principio della conoscenza minima – 1

- è una linea guida di progettazione più che di analisi
 - spesso riferito come Legge di Demetra
- “una classe dovrebbe conoscere solo ciò che le è strettamente necessario”
 - ogni unità di programma dovrebbe conoscere solo poche altre unità di programma strettamente correlate;
 - ogni unità di programma dovrebbe interagire solo con le unità che conosce direttamente.

principio della conoscenza minima – 2

```
public class Car {  
    private Driver d;  
  
    private Vector<Passengers> p;  
  
    public Car(Context context){  
        this.d = context.driver;  
        this.p = context.getPassengers();  
    }  
}
```

principio della conoscenza minima – 2

```
public class Car {  
    private Driver d;  
    private Vector<Passengers> p;  
    public Car(Context context){  
        this.d = context.driver;  
        this.p = context.getPassengers();  
    }  
}
```

- questo esempio viola la legge di Demetra
- Car deve esplicitamente sapere che Context ha :
 - un attributo driver
 - un metodo getPassengers
- questa soluzione impatta sul
 - coupling delle classi del sistema
 - principio di information hiding
- i cambiamenti di Context incidono direttamente su Car, anche se le due classi apparentemente dovrebbero essere scorrelate

principio della conoscenza minima – 3

```
public class Car {  
    private Driver d;  
  
    private Vector<Passengers> p;  
  
    public Car(Context context){  
        this.d = context.driver;  
        this.p = context.getPassengers();  
    }  
}
```

- soluzione appropriata che rispetta la legge di Demetra

```
} public class Car {  
    private Driver d;  
  
    private Vector<Passengers> p;  
  
    public Car(Driver driver){  
        this(driver, null);  
    }  
  
    public Car(Driver driver, Vector<Passengers> listOfPassengers){  
        this.d = driver;  
        this.p = listOfPassengers;  
    }  
}
```


in conclusione RDD

- un modo di pensare alla progettazione O.O.
 - **Responsibility Driven Design**
- gli oggetti software sono definiti in funzione di una loro responsabilità
 - assegnata dal progettista
 - obblighi sulla struttura o sul comportamento
 - responsabilità di fare
 - responsabilità di conoscere
 - responsabilità di collaborare

bibliografia di riferimento

- “Design Patterns – Elementi per il riuso di software a oggetti”, Gamma, Helm, Johnson, Vlissides (GoF). Addison-Wesley (1995).
 - Design Patterns: Elements of Reusable Object-Oriented Software
- “Applicare UML e i Pattern – Analisi e progettazione orientata agli oggetti”, Larman. 3za Edizione. Pearson (2005).
 - “Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development”